

Prática — Gerador de Aulas via WhatsApp com LLM (Claude) e LaTeX

Lucas Augusto Nunes de Barros

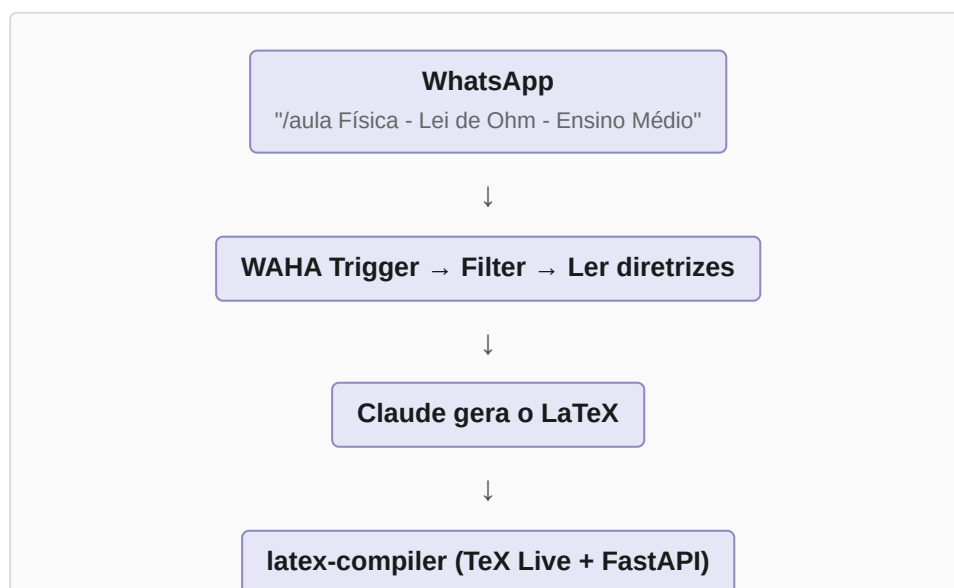
Descrição da atividade

1. Uma stack própria, sem alterar o que já estava validado

Com a base do card12 (n8n + WAHA respondendo mensagens de WhatsApp) já documentada e funcionando, esta prática propôs um passo além: em vez de só ecoar a mensagem recebida, o bot deveria entender um pedido de aula em linguagem natural, gerar o conteúdo com uma LLM e responder com um documento formatado. Para não alterar a infraestrutura já validada, foi criada uma cópia independente do `docker-compose.yml` dentro de `pratica/`, com seus próprios volumes de dados e sua própria sessão de WhatsApp — a stack original permanece intacta como registro histórico do que foi implementado antes.

2. O workflow "Gerador de Aulas"

O workflow foi montado inteiramente através da API REST do próprio n8n — conta de usuário, instalação do nó comunitário da WAHA, credenciais e os nove nós do workflow foram criados via chamadas HTTP diretas, sem depender da interface gráfica. O fluxo funciona assim: um *WAHA Trigger* recebe a mensagem; um *Filter* só deixa passar pedidos que comecem com `/aula`; um nó de leitura de arquivo carrega as diretrizes de formatação de um arquivo de texto; essas diretrizes viram o *system prompt* de uma chamada ao Claude, que gera o LaTeX completo da aula; um serviço local de compilação (TeX Live + FastAPI, também parte desta prática) transforma esse LaTeX em PDF; e, por fim, o PDF é convertido para base64 e enviado de volta pelo WhatsApp — ou, em caso de falha em qualquer etapa, uma mensagem de erro é enviada no lugar.





PDF em base64 → WhatsApp

Fluxo do workflow "Gerador de Aulas"

Dificuldades

O primeiro problema relatado foi que "o n8n não consegue conectar na API do Claude" mesmo com uma chave de API válida cadastrada no node. Testando a mesma credencial isoladamente, através de uma execução direta via API do n8n (sem passar pela interface), a chamada funcionou perfeitamente — o Claude respondeu normalmente. Isso descartou um problema de rede ou de chave inválida: a causa real era que a credencial selecionada no node, dentro da interface, não tinha sido de fato salva junto com o workflow.

Rodando o workflow completo de ponta a ponta, três problemas adicionais apareceram, todos só visíveis em execução real:

O documento gerado pelo Claude terminava cortado no meio de uma seção, sem fechar as chaves nem o `\end{document}`. A causa era o limite padrão de tokens de saída do node (1024), baixo demais para um documento de aula com quatro seções e conteúdo teórico detalhado. Resolvido aumentando esse limite para 8192.

```
// antes: options.maxTokens ausente (padrão 1024)
// depois:
options.maxTokens = 8192
```

Depois de corrigido o corte, uma segunda tentativa revelou que o Claude, mesmo instruído explicitamente a não fazer isso, embrulhou o LaTeX inteiro em um bloco de código Markdown (````latex ... ````). O pdf_{LaTeX} travava logo na primeira linha, tentando interpretar as três crases como comando. Em vez de insistir apenas na instrução do prompt, o node de compilação passou a remover esses blocos de forma defensiva antes de enviar o conteúdo ao compilador — não é seguro depender de um LLM seguir uma instrução de formatação 100% das vezes.

```
tex = $json.content[0].text
  .replace(/^```(latex)?\s*\n?/, '')
  .replace(/\n?```\s*$/, '')
```

Por fim, com o PDF já compilado corretamente, o envio pelo WhatsApp falhava com o erro *"Either 'file.url' or 'file.data' must be specified"*. A tentativa inicial de acessar o conteúdo binário diretamente por expressão (`$binary.pdf.data`) não resolvia para uma string válida. A solução foi adicionar um node dedicado de conversão binário→base64 antes do envio, configurado explicitamente para preservar os bytes como base64 em vez de tentar interpretá-los como JSON — um modo de configuração desse node especificamente para esse fim, diferente do modo padrão.

Conclusões

Construir o workflow inteiro via chamadas diretas à API do n8n — em vez de montá-lo manualmente na interface — permitiu um ciclo de diagnóstico muito mais rápido: cada execução de teste podia ser inspecionada nó a nó, com o payload exato que cada um recebeu e produziu, o que foi decisivo para encontrar os três bugs reais que só apareciam em execução (não em uma revisão estática da configuração). Essa prática também reforçou um princípio geral ao integrar LLMs em pipelines automatizados: instruções de formatação no prompt (como "responda só com o código, sem blocos de markdown") reduzem a chance de erro, mas não a eliminam — o pipeline precisa lidar defensivamente com desvios do formato esperado, em vez de assumir que o modelo sempre vai obedecer à risca.

Referências

[1] ANTHROPIC. Messages API Reference.

Disponível em: <https://docs.anthropic.com/en/api/messages>

Acesso em 06 de agosto de 2026

[2] N8N.IO. Anthropic node documentation.

Disponível em: <https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-langchain.anthropic/>

Acesso em 06 de agosto de 2026

[3] WAHA (WHATSAPP HTTP API). Documentação oficial.

Disponível em: <https://waha.devlike.pro/>

Acesso em 06 de agosto de 2026

[4] FASTAPI. Documentação oficial — framework usado no serviço local de compilação LaTeX.

Disponível em: <https://fastapi.tiangolo.com/>

Acesso em 06 de agosto de 2026

[5] TEX USERS GROUP. TeX Live — distribuição LaTeX usada no serviço de compilação.

Disponível em: <https://tug.org/texlive/>

Acesso em 06 de agosto de 2026